无锁并发队列的设计

Posted on 2024-01-09 09:32:27 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2ahHZgMOCIfy9TnBOaS8pXKjf5n)

简介

前文介绍了无锁并发栈的设计，本文继续介绍无锁队列的设计。队列和栈容器的难点稍微不同，因为对于队列结构，push()和pop()分别访问其不同部分，而在栈容器上，这两项操作都访问头节点，所以两种数据结构所需的同步操作相异。如果某线程在队列一端做出改动，而另一线程同时访问队列另一端，代码就要保证前者的改动过程能正确地为后者所见

单一消费者和生产者队列

我们实现一个简单的无锁队列，只应对一个生产者一个消费者的情况，便于我们理解

```
#include<atomic>
#include<memory>

template<typename T>
class SinglePopPush
{
private:
    struct node
    {
        std::shared_ptr<T> data;
        node* next;
        node() :
            next(nullptr)
        {}
    };
    std::atomic<node*> head;
    std::atomic<node*> tail;
    node* pop_head()
    {
        node* const old_head = head.load();
        // <--- - 1
        if (old_head == tail.load())
        {
            return nullptr;
        }
        head.store(old_head->next);
        return old_head;
    }
public:
    SinglePopPush() :
        head(new node), tail(head.load())
    {}
    SinglePopPush(const SinglePopPush& other) = delete;
    SinglePopPush& operator=(const SinglePopPush& other) = delete;
    ~SinglePopPush()
    {
        while (node* const old_head = head.load())
        {
            head.store(old_head->next);
            delete old_head;
        }
    }
```

```
43.    std::shared_ptr<T> pop()
44.    {
45.        node* old_head = pop_head();
46.        if (!old_head)
47.        {
48.            return std::shared_ptr<T>();
49.        }
50.        // ⇐--- -2
51.        std::shared_ptr<T> const res(old_head->data);
52.            delete old_head;
53.        return res;
54.    }
55.    void push(T new_value)
56.    {
57.        std::shared_ptr<T> new_data(std::make_shared<T>(new_value));
58.        // ⇐--- - 3
59.        node* p = new node;
60.        //⇐--- - 4
61.        node* const old_tail = tail.load();
62.        //⇐--- - 5
63.        old_tail->data.swap(new_data);
64.        //⇐--- - 6
65.        old_tail->next = p;
66.        //⇐--- - 7
67.        tail.store(p);
68.    }
69. };
```

上面的实现初看上去还不错。在同一时刻，如果只有一个线程调用push()，且仅有一个线程调用pop()，这份代码便可以相对完美地工作。

本例中的push()和pop()之间存在先行关系，这点很重要，它使队列的使用者可安全地获取数据。

tail指针的存储操作7与其载入操作1同步：按控制流程，在运行push()的线程上，原有的尾节点中的data指针先完成存储操作5，然后tail才作为指针存入新值7；

并且，在运行pop()的线程上，tail指针先完成载入操作1，原来的data指针才执行加载操作2，故data的存储操作5在载入操作1之前发生

（全部环节正确无误。因此这个单一生产者、单一消费者（Single-Producer Single-Consumer，SPSC）队列可以完美地工作。

不过，若多个线程并发调用push()或并发调用pop()，便会出问题。我们先来分析push()。如果有两个线程同时调用push()，就会分别构造一个新的空节点并分配内存3，而且都从tail指针读取相同的值4，结果它们都针对同一个尾节点更新其数据成员，却各自把data指针和next指针设置为不同的值5和6。这形成了数据竞争！

pop_head()也有类似问题，若两个线程同时调用这个函数，它们就会读取同一个头节点而获得相同的next指针，而且都把它赋予head指针以覆盖head指针原有的值。最终两个线程均认为自己获取了正确的头节点，这是错误的根源。给定一项数据，我们不仅要确保仅有一个线程可对它调用pop()，如果有别的线程同时读取头节点，则还需保证它们可以安全地访问头节点中的next指针。我们曾在前文的无锁栈容器中遇见过类似问题，其pop()函数也有完全相同的问题。

多线程push

解决多线程push的竞争问题。

一种方法是将data指针原子化，通过比较-交换操作来设置它的值。如果比较-交换操作成功，所操作的节点即为真正的尾节点，我们便可安全地设定next指针，使之指向新节点。若比较-交换操作失败，就表明有另一线程同时存入了数据，我们应该进行循环，重新读取tail指针并从头开始操作。

如果std::shared_ptr<>上的原子操作是无锁实现，那便万事大吉，否则我们仍需采取别的方法。一种可行的方法是令pop()返回std::unique_ptr<>指针（凭此使之成为指涉目标对象的唯一指针），并在队列中存储指向数据的普通指针。这样让代码得以按std::atomic<T*>的形式存储指针，支持必要的compare_exchange_strong()调用。

```
1. void push(T new_value)
2. {
3.     std::unique_ptr<T> new_data(new T(new_value));
4.     counted_node_ptr new_next;
5.     new_next.ptr=new node;
6.     new_next.external_count=1;
7.     for(;;)
8.     {
9.         //<---- 1
10.        node* const old_tail=tail.load();
11.        T* old_data=nullptr;
12.        //<---- 2
13.        if(old_tail->data.compare_exchange_strong(
14.            old_data,new_data.get()))
15.        {
16.            old_tail->next=new_next;
17.            // 3
18.            tail.store(new_next.ptr);
19.            new_data.release();
20.            break;
21.        }
22.    }
23. }
```

引用计数避免了上述的数据竞争，但那不是push()中仅有的数据竞争。只要我们仔细观察，便会发现其代码模式与栈容器相同：先载入原子指针1，然后依据该指针读取目标值2。

另一线程有可能同时更新tail指针3，如果该更新在pop()内部发生，最终将导致删除尾节点。若尾节点先被删除，代码却依然根据指针读取目标值，就会产生未定义行为。

有一种方法能解决上面的问题，且该方法颇具吸引力：在尾节点中添加一外部计数器，与处理头节点的方法相同。不过队列中的每个节点已配备一个外部计数器，分别存储在对应前驱节点内的next指针中。

若要让同一个节点具有两个外部计数器，便需要改动引用计数的实现方式，以免过早删除节点。我们为了满足上述要求，可在节点的结构体中记录外部计数器的数目，外部计数器一旦发生销毁，该数目则自减，并且将该外部计数器的值加到内部计数器的值之上。对于任意特定节点，如果内部计数器的值变为零，且再也没有外部计数器存在，我们就知道该节点能被安全地删除.

```
1. template<typename T>
2. class lock_free_queue
3. {
4. private:
5.     struct node;
6.     struct counted_node_ptr
7.     {
8.         int external_count;
9.         node* ptr;
10.    };
11.    std::atomic<counted_node_ptr> head;
12.    //<---- 1
13.    std::atomic<counted_node_ptr> tail;
14.    struct node_counter
15.    {
16.        unsigned internal_count:30;
17.        //<---- 2
18.        unsigned external_counters:2;
19.    };
20.    struct node
21.    {
22.        std::atomic<T*> data;
23.        //<----  3
24.        std::atomic<node_counter> count;
25.        counted_node_ptr next;
26.        node()
27.        {
28.            node_counter new_count;
29.            new_count.internal_count=0;
30.            //<----  4
31.            new_count.external_counters=2;
32.            count.store(new_count);
33.
34.            next.ptr=nullptr;
35.            next.external_count=0;
36.        }
37.    };
38. public:
39.    void push(T new_value)
40.    {
41.        std::unique_ptr<T> new_data(new T(new_value));
42.        counted_node_ptr new_next;
```

```
43.       new_next.ptr=new node;
44.       new_next.external_count=1;
45.       counted_node_ptr old_tail=tail.load();
46.       for(;;)
47.       {
48.          // 5
49.          increase_external_count(tail,old_tail);
50.          T* old_data=nullptr;
51.          // 6
52.          if(old_tail.ptr->data.compare_exchange_strong(
53.            old_data,new_data.get()))
54.          {
55.             old_tail.ptr->next=new_next;
56.             old_tail=tail.exchange(new_next);
57.             // 7
58.             free_external_counter(old_tail);
59.             new_data.release();
60.             break;
61.          }
62.          old_tail.ptr->release_ref();
63.       }
64.    }
65. };
```

tail指针(1处) 和head指针的型别均为atomic<counted_node_ptr>，而node结构体则以成员count (3处)取代原有的internal_count。

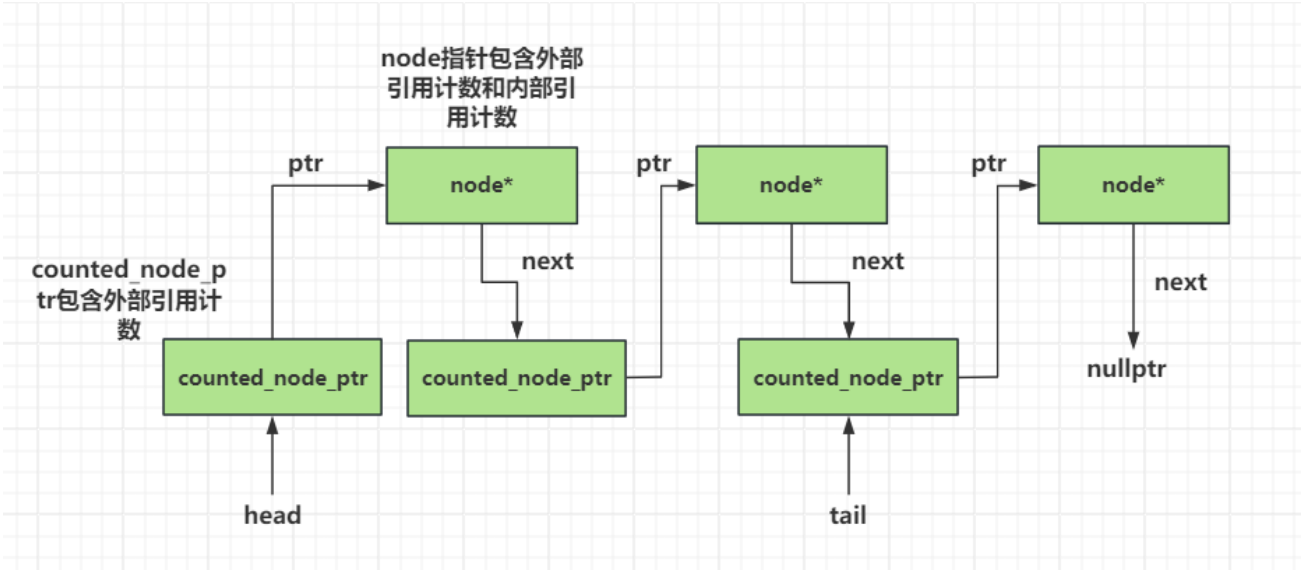
该count成员也是一个结构体，内含internal_count变量和新引入的external_counters变量(2处) 。请注意，external_counters仅需使用两位，因为同一个节点最多只可能有两个外部计数器。因此，结构体count为它分配了一个两位的位域，而把internal_count设定为30位的整型值，从而维持了计数器32位的整体尺寸。

按此处理，内部计数器的取值范围仍然非常大，还保证了在32位或64位计算机上，一个机器字（machine word）便能容纳整个结构体。后文很快会解释，为了杜绝条件竞争，上述两种计数器必须合并，视作单一数据项，共同进行更新。只要把结构体的大小限制在单个机器字内，那么在许多硬件平台上，其原子操作就更加有机会以无锁方式实现。

节点经过初始化，其internal_count成员被置零，而external_counters成员则设置成2(4处)，因为我们向队列加入的每个新节点，它最初既被tail指针指涉，也被前一个节点的next指针指涉。

我们先调用一个新函数increase_external_count()令外部计数器的值增加（5处），再载入tail指针，进而读取尾节点的data成员并对它调用compare_exchange_strong()（6处），然后对原有的tail指针执行free_external_counter()（7处）。

我们画一下这个图



多线程pop

多线程pop实现和之前无锁栈类似，我们只要做外部引用计数的增加和内部引用计数的减少即可

```
template<typename T>
class lock_free_queue
{
private:
    struct node
    {
        void release_ref();
        //node的余下代码与代码清单7.16相同
    };
public:
    std::unique_ptr<T> pop()
    {
        // 1
        counted_node_ptr old_head=head.load(std::memory_order_relaxed);
        for(;;)
        {
            //2
            increase_external_count(head,old_head);
            node* const ptr=old_head.ptr;
            if(ptr==tail.load().ptr)
            {
                //3
                ptr->release_ref();
                return std::unique_ptr<T>();
            }
            // 4
            if(head.compare_exchange_strong(old_head,ptr->next))
            {
                T* const res=ptr->data.exchange(nullptr);
                // 5
                free_external_counter(old_head);
                return std::unique_ptr<T>(res);
            }
            // 6
            ptr->release_ref();
        }
    }
};
```

节点的弹出操作从加载old_head指针开始（1处），接着进入一个无限循环，并且令已加载好的指针上的外部计数器的值自增（2处）。若头节点正巧就是尾节点，即表明队列内没有

数据，我们便释放引用（3处），并返回空指针。

否则表明队列中存在数据，因此当前线程试图调用compare_exchange_strong()将其收归己有（4处）。以上调用会对比结构体head和old_head，其成员都包括外部计数器和指针，但均被视作一个整体。无论哪个成员发生了变化而导致不匹配，代码即释放引用（6处）并重新循环。

如果比较-交换操作成功，当前线程就顺利地将节点所属的数据收归己有，故我们随即释放弹出节点的外部计数器（5处），再将数据返回给pop()的调用者。若两个外部计数器都被释放，且内部计数器值变为0，则节点本身可被删除。有几个函数负责处理引用计数

下面是减少引用计数的函数

```
template<typename T>
class lock_free_queue
{
private:
    struct node
    {
        void release_ref()
        {
            node_counter old_counter=
                count.load(std::memory_order_relaxed);
            node_counter new_counter;
            do
            {
                new_counter=old_counter;
                //1
                --new_counter.internal_count;
            }
            //2
            while(!count.compare_exchange_strong(
                  old_counter,new_counter,
                  std::memory_order_acquire,std::memory_order_relaxed));
            if(!new_counter.internal_count &&
               !new_counter.external_counters)
            {
                //3
                delete this;
            }
        }
    };
};
```

尽管我们在这里只改动位域成员internal_count(1处)，也必须按原子化方式更新整个计数器结构体。所以更新操作要用比较-交换函数配合循环实现（2处）。

当计数器internal_count完成自减后，如果内外两个计数器的值均为0，就表明调用release_ref()的是最后一个指涉目标节点的指针（代码清单pop （5 6两处）的ptr），我们应当删除节点（3处）。

接下来我们实现增加引用计数的操作

```
template<typename T>
class lock_free_queue
{
private:
    static void increase_external_count(
        std::atomic<counted_node_ptr>& counter,
        counted_node_ptr& old_counter)
    {
        counted_node_ptr new_counter;
        do
        {
            new_counter=old_counter;
            ++new_counter.external_count;
        }
        while(!counter.compare_exchange_strong(
              old_counter,new_counter,
              std::memory_order_acquire,std::memory_order_relaxed));
        old_counter.external_count=new_counter.external_count;
    }
};
```

increase_external_count()已改成了静态成员函数，需要更新的目标不再是自身固有的成员计数器，而是一个外部计数器，它通过第一个参数传入函数以进行更新。

针对无锁队列的节点释放其外部计数器

```
1. template<typename T>
2. class lock_free_queue
3. {
4. private:
5.     static void free_external_counter(counted_node_ptr &old_node_ptr)
6.     {
7.         node* const ptr=old_node_ptr.ptr;
8.         int const count_increase=old_node_ptr.external_count-2;
9.         node_counter old_counter=
10.            ptr->count.load(std::memory_order_relaxed);
11.        node_counter new_counter;
12.        do
13.        {
14.            new_counter=old_counter;
15.            //<---- 1
16.            --new_counter.external_counters;
17.            //<---- 2
18.            new_counter.internal_count+=count_increase;
19.        }
20.        //<---- 3
21.        while(!ptr->count.compare_exchange_strong(
22.              old_counter,new_counter,
23.              std::memory_order_acquire,std::memory_order_relaxed));
24.        if(!new_counter.internal_count &&
25.           !new_counter.external_counters)
26.        {
27.            //<---- 4
28.            delete ptr;
29.        }
30.    }
31. };
```

与free_external_counter()对应的是increase_external_count()函数，该函数对整个计数器结构体仅执行一次compare_exchange_strong()，便合并更新了其中的两个计数器(3处)，这与release_ref()中更新internal_count的自减操作类似。

计数器external_counters则同时自减(1处)。如果这两个值均变为0，就表明目标节点再也没有被指涉，遂可以安全删除（4处）。

为了避免条件竞争，上述更新行为需要整合成单一操作完成，因此需要用比较-交换函数配合循环运行。若两项更新分别独立进行，万一有两个线程同时调用该函数，则它们可能都会认

为自己是最后的执行者，所以都删除节点，结果产生未定义行为。

优化

虽然上述代码尚可工作，也无条件竞争，但依然存在性能问题。一旦某线程开始执行 push() 操作，针对 old_tail.ptr->data成功完成了compare_exchange_strong()调用(push代码6处)，就没有其他线程可以同时运行push()。若有其他任何线程试图同时压入数据，便始终看不到nullptr，而仅能看到上述线程执行push()传入的新值，导致compare_exchange_strong()调用失败，最后只能重新循环。这实际上是忙等，消耗CPU周期却一事无成，结果形成了实质的锁。第一个push()调用令其他线程发生阻塞，直到执行完毕才解除，所以这段代码不是无锁实现。问题不止这一个。若别的线程被阻塞，则操作系统会提高对互斥持锁的线程的优先级，好让它尽快完成，但本例却无法依此处理，被阻塞的线程将一直消耗CPU周期，等到最初调用push()的线程执行完毕才停止。这个问题带出了下一条妙计：让等待的线程协助正在执行push()的线程，以实现无锁队列。

我们很清楚应该在这种方法中具体做什么：先设定尾节点上的next指针，使之指向一个新的空节点，且必须随即更新tail指针。由于空节点全都等价，因此这里所用空节点的起源并不重要，其创建者既可以是成功压入数据的线程，也可以是等待压入数据的线程。如果将节点内的next指针原子化，代码就能借compare_exchange_strong()设置其值。只要设置好了next指针，便可使用compare_exchange_weak()配合循环设定tail指针，借此令它依然指向原来的尾节点。若tail指针有变，则说明它已同时被别的线程更新过，因此我们停止循环，不再重试。

pop()需要稍微改动才可以载入原子化的next指针

```
1. template<typename T>
2. class lock_free_queue
3. {
4. private:
5.     struct node
6.     {
7.         std::atomic<T*> data;
8.         std::atomic<node_counter> count;
9.         //<----  1
10.        std::atomic<counted_node_ptr> next;
11.    };
12. public:
13.    std::unique_ptr<T> pop()
14.    {
15.        counted_node_ptr old_head=head.load(std::memory_order_relaxed);
16.        for(;;)
17.        {
18.            increase_external_count(head,old_head);
19.            node* const ptr=old_head.ptr;
20.            if(ptr==tail.load().ptr)
21.            {
22.                return std::unique_ptr<T>();
23.            }
24.            //  <----  2
25.            counted_node_ptr next=ptr->next.load();
26.            if(head.compare_exchange_strong(old_head,next))
27.            {
28.                T* const res=ptr->data.exchange(nullptr);
29.                free_external_counter(old_head);
30.                return std::unique_ptr<T>(res);
31.            }
32.            ptr->release_ref();
33.        }
34.    }
35. };
```

上面的代码进行了简单改动：next指针现在采用了原子变量（1处），并且（2处）的载入操作也成了原子操作。本例使用了默认的memory_order_seq_cst次序，而ptr->next指针原本属于std::atomic<counted_node_ptr>型别，在（2 处）隐式转化成counted_node_ptr型别，这将触发原子化的载入操作，故无须显式调用load()。不过我们还是进行了显式调用，目的是提醒自己，在以后优化时此处应该显式设定内存次序。

新版本的push()相对更复杂，如下

```
1. template<typename T>
2. class lock_free_queue
3. {
4. private:
5.     // ⇐---- 1
6.     void set_new_tail(counted_node_ptr &old_tail,
7.                       counted_node_ptr const &new_tail)
8.     {
9.         node* const current_tail_ptr=old_tail.ptr;
10.        // ⇐---- 2
11.        while(!tail.compare_exchange_weak(old_tail,new_tail) &&
12.              old_tail.ptr==current_tail_ptr);
13.        // ⇐---- 3
14.        if(old_tail.ptr==current_tail_ptr)
15.            //⇐---- 4
16.            free_external_counter(old_tail);
17.        else
18.            //⇐---- 5
19.            current_tail_ptr->release_ref();
20.    }
21. public:
22.    void push(T new_value)
23.    {
24.        std::unique_ptr<T> new_data(new T(new_value));
25.        counted_node_ptr new_next;
26.        new_next.ptr=new node;
27.        new_next.external_count=1;
28.        counted_node_ptr old_tail=tail.load();
29.        for(;;)
30.        {
31.            increase_external_count(tail,old_tail);
32.            T* old_data=nullptr;
33.            //⇐---- 6
34.            if(old_tail.ptr->data.compare_exchange_strong(
35.                  old_data,new_data.get()))
36.            {
37.                counted_node_ptr old_next={0};
38.                //⇐---- 7
39.                if(!old_tail.ptr->next.compare_exchange_strong(
40.                      old_next,new_next))
41.                {
42.                    //⇐---- 8
```

```
43.                 delete new_next.ptr;
44.                 new_next=old_next;   // <---- 9
45.             }
46.             set_new_tail(old_tail, new_next);
47.             new_data.release();
48.             break;
49.         }
50.         else    // <---- 10
51.         {
52.             counted_node_ptr old_next={0};
53.             // <---- 11
54.             if(old_tail.ptr->next.compare_exchange_strong(
55.                   old_next,new_next))
56.             {
57.                 // <---- 12
58.                 old_next=new_next;
59.                 // <---- 13
60.                 new_next.ptr=new node;
61.             }
62.             // <---- 14
63.             set_new_tail(old_tail, old_next);
64.         }
65.     }
66. }
67. };
```

由于我们确实想在(6处)设置data指针，而且还需接受另一线程的协助，因此引入了else分支以处理该情形(10处)。上述push()的新版本先在(6处)处设置好节点内的data指针，然后通过compare_exchange_strong()更新next指针(7处)，从而避免了循环。

若交换操作失败，我们便知道另一线程同时抢先设定了next指针，遂无须保留函数中最初分配的新节点，可以将它删除（8处）。

虽然next指针是由别的线程设定的，但代码依然持有其值，留待后面更新tail指针（9处）。更新tail指针的代码被提取出来，写成set_new_tail()函数（1处）。它通过compare_exchange_weak()配合循环来更新tail指针（2处）。

如果其他线程试图通过push()压入新节点，计数器external_count就会发生变化，而上述新函数正是为了防止错失这一变化。但我们也要注意，若另一线程成功更新了tail指针，其值便不得再次改变。若当前线程重复更新tail指针，便会导致控制流程在队列内部不断循环，这种做法完全错误。

相应地，如果比较-交换操作失败，所载入的ptr指针也需要保持不变。在脱离循环时，假如ptr指针的原值和新值保持一致（3处）就说明tail指针的值肯定已经设置好，原有的外部计数器则需要释放（4处）。若ptr指针前后有所变化，则另一线程将释放计数器，而当前线程要释放它持有的唯一一个tail指针（5处）。

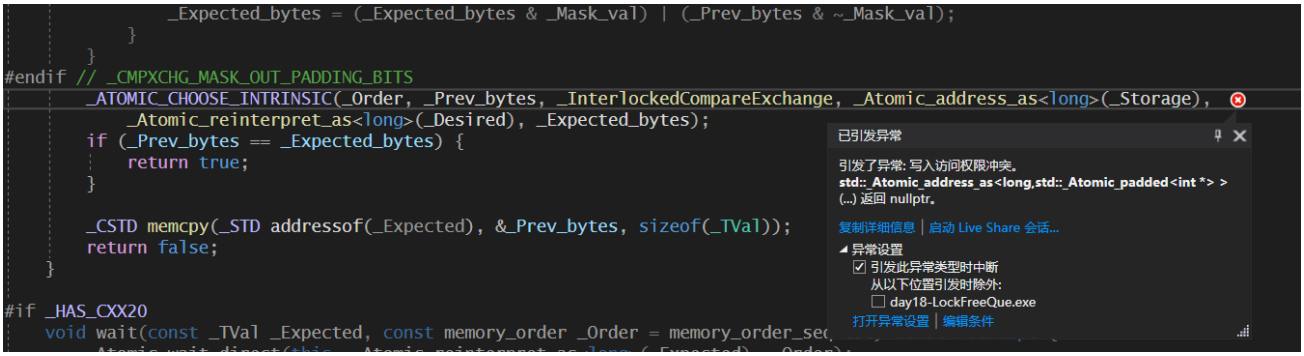
这里，若多个线程同时调用push()，那么只有一个线程能成功地在循环中设置data指针，失败的线程则转去协助成功的线程完成更新。当前线程一进入push()就分配了一个新节点，我们先更新next指针，使之指向该节点（11处）。假定操作成功，该节点就充当新的尾节点⑫，而我们还需另行分配一个新节点，为下一个压入队列的数据预先做好准备⑬。接着，代码尝试调用set_new_tail()以设置尾节点（14处），再重新循环。

官方案例的隐患

我们基于上面的案例执行下面的测试代码，发现程序崩溃

```
1. void TestCrushQue() {
2.     crush_que<int>  que;
3.     std::thread t1([&]() {
4.         for (int i = 0; i < TESTCOUNT*10000; i++) {
5.             que.push(i);
6.             std::cout << "push data is " << i << std::endl;
7.             std::this_thread::sleep_for(std::chrono::milliseconds(10));
8.         }
9.         });
10.
11.
12.
13.     std::thread t2([&]() {
14.         for (int i = 0; i < TESTCOUNT*10000;) {
15.             auto p = que.pop();
16.             if (p == nullptr) {
17.                 std::this_thread::sleep_for(std::chrono::milliseconds(10));
18.                 continue;
19.             }
20.             i++;
21.             std::cout << "pop data is " << *p << std::endl;
22.         }
23.         });
24.
25.     t1.join();
26.     t2.join();
27.
28. }
```

我们看到崩溃在底层代码的原子变量交换这里



我们按照调用堆栈往上查找，发现是head和tail的ptr为空导致

```
//<----  6
if (old_tail.ptr->data.compare_exchange_strong(
    old_data, new_data.get()))
{
    counted_node_ptr old_next = {0};
    counted_node_ptr now_next = old_tail.ptr->next.load();
    //<----  7
    if (!old_tail.ptr->next.compare_exchange_strong(
        old_next, new_next))
    {
        //<----  8
        delete new_next.ptr;
        new_next = old_next;    // <----  9
    }
    set_new_tail(old_tail, new_next);
    new_data.release();
    break;
}
```

解决这个问题比较简单，我们在队列的构造函数中添加head和tail的初始化即可。

```
memoryleak_que() {
    counted_node_ptr new_next;
    new_next.ptr = new node();
    new_next.external_count = 1;
    tail.store(new_next);
    head.store(new_next);
    std::cout << "new_next.ptr is " << new_next.ptr << std::endl;
}
```

我们也需要在析构函数里回收头尾节点，基本思路是依次出队，但是因为最后一个节点为tail，当head和tail相等时则停止回收，所以我们要额外回收头部节点(此时头部和尾部节点重合)

```
~memoryleak_que() {
    while (pop());
    auto head_counted_node = head.load();
    delete head_counted_node.ptr;
}
```

为了测试内存泄漏，我们在栈中添加一个静态成员变量

```
class memoryleak_que{
public:
static std::atomic<int> destruct_count;
};

template<typename T>
std::atomic<int> lock_free_queue<T>::destruct_count = 0;
```

我们在release_ref和free_external_counter中删除指针时增加这个静态成员变量的数量，最后统计删除的数量和我们开辟的数量是否相等

```
void release_ref()
{
    std::cout << "call release ref " << std::endl;
    node_counter old_counter =
    count.load(std::memory_order_relaxed);
    node_counter new_counter;
    do
    {
        new_counter = old_counter;
        //1
        --new_counter.internal_count;
    }
        //2
    while (!count.compare_exchange_strong(
            old_counter, new_counter,
            std::memory_order_acquire, std::memory_order_relaxed));
        if (!new_counter.internal_count &&
            !new_counter.external_counters)
    {
        //3
        delete this;
        std::cout << "release_ref delete success" << std::endl;
        destruct_count.fetch_add(1);
    }
}
```

```
static void free_external_counter(counted_node_ptr& old_node_ptr)
{
    std::cout << "call  free_external_counter " << std::endl;
    node* const ptr = old_node_ptr.ptr;
    int const count_increase = old_node_ptr.external_count - 2;
    node_counter old_counter =
        ptr->count.load(std::memory_order_relaxed);
    node_counter new_counter;
    do
    {
        new_counter = old_counter;
        //<---- 1
        --new_counter.external_counters;
        //<---- 2
        new_counter.internal_count += count_increase;
    }
    //<---- 3
    while (!ptr->count.compare_exchange_strong(
            old_counter, new_counter,
            std::memory_order_acquire, std::memory_order_relaxed));
    if (!new_counter.internal_count &&
            !new_counter.external_counters)
    {
        //<---- 4
        destruct_count.fetch_add(1);
        std::cout << "free_external_counter delete success" << std::endl;
        delete ptr;
    }

}
```

测试并发执行两个线程，最后assert断言删除节点数和开辟的节点数相等

```
1. void TestLeakQue() {
2.     memoryleak_que<int>  que;
3.     std::thread t1([&]() {
4.         for (int i = 0; i < TESTCOUNT ; i++) {
5.             que.push(i);
6.             std::cout << "push data is " << i << std::endl;
7.             std::this_thread::sleep_for(std::chrono::milliseconds(10));
8.         }
9.         });
10.
11.
12.
13.     std::thread t2([&]() {
14.         for (int i = 0; i < TESTCOUNT ;) {
15.             auto p = que.pop();
16.             if (p == nullptr) {
17.                 std::this_thread::sleep_for(std::chrono::milliseconds(10));
18.                 continue;
19.             }
20.             i++;
21.             std::cout << "pop data is " << *p << std::endl;
22.         }
23.         });
24.
25.     t1.join();
26.     t2.join();
27.
28.     assert(que.destruct_count == TESTCOUNT );
29.
30. }
```

测试触发断言，说明存在内存泄漏。

经过调试我们发现其实是在pop头部节点时判断head和tail相等，直接返回空指针，但是引用计数没有做减少。这和栈的方式不同，栈的pop判断条件如果head节点的ptr指向空地址，说明这个节点为无效节点无需pop直接返回空指针，当有新数据插入时在头部插入新节点并更新head为新节点。这么做保证了即使最后那个无效节点引用计数怎么增加都无所谓。

但是队列不行，队列的操作方式是先开辟了head和tail节点，这两个节点最开始是无效的，但是当插入数据时，就将head的ptr指向的数据data更新为新的数据即可。这样head之前和tail相等时pop增加的引用计数如果不合理减少就会造成问题。

解决的思路也比较简单，如果head和tail相等说明为空队列，空队列减少该节点内部引用计数即可。

```
1.  std::unique_ptr<T> pop()
2. {
3.     counted_node_ptr old_head = head.load(std::memory_order_relaxed);
4.     for (;;)
5.     {
6.         increase_external_count(head, old_head);
7.         node* const ptr = old_head.ptr;
8.         if (ptr == tail.load().ptr)
9.         {
10.            //头尾相等说明队列为空，要减少内部引用计数
11.            ptr->release_ref();
12.            return std::unique_ptr<T>();
13.        }
14.        //  <---  2
15.        counted_node_ptr next = ptr->next.load();
16.        if (head.compare_exchange_strong(old_head, next))
17.        {
18.            T* const res = ptr->data.exchange(nullptr);
19.            free_external_counter(old_head);
20.            return std::unique_ptr<T>(res);
21.        }
22.        ptr->release_ref();
23.    }
24. }
```

最后我们测试多线程pop和push的情况，目前稳定回收节点并且并发安全

```
1. void TestLockFreeQueMultiPushPop() {
2.     lock_free_queue<int>  que;
3.     std::thread t1([&]() {
4.         for (int i = 0; i < TESTCOUNT * 100; i++) {
5.             que.push(i);
6.             std::cout << "push data is " << i << std::endl;
7.             std::this_thread::sleep_for(std::chrono::milliseconds(10));
8.         }
9.         });
10.
11.    std::thread t4([&]() {
12.        for (int i = TESTCOUNT*100; i < TESTCOUNT * 200; i++) {
13.            que.push(i);
14.            std::cout << "push data is " << i << std::endl;
15.            std::this_thread::sleep_for(std::chrono::milliseconds(10));
16.        }
17.        });
18.
19.    std::thread t2([&]() {
20.        for (int i = 0; i < TESTCOUNT * 100;) {
21.            auto p = que.pop();
22.            if (p == nullptr) {
23.                std::this_thread::sleep_for(std::chrono::milliseconds(10));
24.                continue;
25.            }
26.            i++;
27.            std::cout << "pop data is " << *p << std::endl;
28.        }
29.        });
30.
31.    std::thread t3([&]() {
32.        for (int i = 0; i < TESTCOUNT * 100;) {
33.            auto p = que.pop();
34.            if (p == nullptr) {
35.                std::this_thread::sleep_for(std::chrono::milliseconds(10));
36.                continue;
37.            }
38.            i++;
39.            std::cout << "pop data is " << *p << std::endl;
40.        }
41.        });
42.
```

```
43.     t1.join();
44.     t2.join();
45.     t3.join();
46.     t4.join();
47.     assert(que.destruct_count == TESTCOUNT * 200);
48. }
```

总结

本文介绍了无锁队列的实现，利用了引用计数的思想，实现了并发安全的无锁队列。

源码链接：

https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day18-LockFreeQue (https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day18-LockFreeQue)

视频链接：

https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290)

浏览(2240) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(535) (JavaScript:void(0);)

热门评论

请输入正文

评论

没有更多评论了,点击收起 ^ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托

了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)